Week 1 Design Opportunity: Per-User Isolation for Conversational Memory

Deliverable 2, Week 1 review

Component: Privacy, safety, and isolation

Presenter: Asfiya Mukthasar

---

1. The problem

Requirement R16: one user's memories must never be retrieved in another user's session.

Failure X3, constructed in Deliverable 1: two users of a coding assistant work on similar stacks and describe problems in similar words. User A reports a database connection failure; A's environment details, including a connection string, are stored. User B later reports a similar failure. Retrieval ranked by semantic similarity has no property that prevents A's records from scoring highly against B's query, because the content genuinely is similar.

The failure is silent. By the only measure the system applies, the retrieved record is relevant.

This maps to a non-negotiable evaluation gate: cross-tenant isolation must be tested.

---

2. Why the current approach does not solve it

Current approach: rank stored memories by similarity to the query, return the top K.

There is no owner constraint anywhere in that path. Nothing distinguishes A's memory from B's except content, and content is what makes them match.

---

3. The finding: the index cannot isolate for you

Both mainstream index families actively group the data that isolation must separate.

FAISS partitions vectors into inverted lists using a coarse quantizer, and a query scans only selected lists [10, §2]. That establishes subset-restricted search as structural. But assignment to a list is by geometric similarity, so two users' similar memories land in the same list.

HNSW is stronger in the same direction: it links semantically similar nodes directly as graph neighbours [11, §4], and its neighbour-selection heuristic exists specifically to perform well on clustered data. Per-user memories about common problems form exactly such clusters.

Conclusion: isolation cannot be delegated to the index. It must be enforced above it.

---

4. Proposed change

Attach an owner identifier to every stored memory at write time. Constrain every retrieval so that only memories owned by the querying user are eligible.

Simple, and it eliminates X3 by construction: another user's records are never candidates.

---

5. Cost and risk

Integration cost: low. One attribute at write, one constraint at read.

The real cost is a dependency it creates: every memory must carry a correct owner ID at admission, which places a correctness requirement on the write path (linking to R17).

Storage consideration: HNSW index overhead is roughly 60 to 450 bytes per object [11, §4.2.3]. This matters for the alternative below.

---

6. The open question, and why it is not settled

Where the filter is applied changes the guarantee.

Filter during search: other users' vectors are never examined. Strongest guarantee. But in HNSW, skipping non-matching nodes mid-traversal risks stranding the search in a region of ineligible nodes, degrading recall. Search always begins from a single shared top-layer entry point [11, §4], so there is no per-user entry into the structure.

Filter after search: other users' vectors were examined and then discarded. The user never sees them, but the system touched them. Weaker guarantee, no recall risk.

Alternative: one index per user. Clean isolation and clean deletion, no filtering logic. Costs index overhead per user and loses any cross-user structure, which for this domain is probably not wanted anyway.

Neither source settles this, because neither addresses multi-tenancy at all. This is a system design decision, not a documented property.

7. Would something simpler work?

Per-user indexes are arguably simpler than filtered shared search: no filter logic, and deletion becomes trivial. The trade is memory overhead. This is a genuine alternative, not an obviously worse one, and is included in the validation below.

---

8. Validation plan

Smallest useful experiment: create two users with deliberately near-identical memories (same stack, same error, different credentials). Issue queries from each user that would match the other's records on similarity alone.

Success criterion: zero cross-user records returned, across all queries, for both filter placements.

Also measured: recall against a known ground-truth set for each user, and retrieval latency, comparing filter-during, filter-after, and per-user indexes.

Evidence that would falsify the proposal: any filtered query returning another user's record. Separately, if filter-during degrades recall materially on clustered data, the shared-index approach is rejected in favour of per-user indexes.

---

9. Decision

Adopt: owner-ID filtering as the isolation mechanism for R16.

Prototype: filter placement (during vs after search) and shared-index vs per-user indexes, resolved by the experiment above.

Related decisions this week: machine unlearning rejected as a deletion mechanism (targets model weights; memory is external). Admission-time PII detection marked prototype (necessary for R17, but no source reports accuracy on debugging content). Integrity against injection deferred to the component 7 scan.

---

10. Open questions

Deletion has an implementation-dependent caveat: HNSW lists element removal as unsupported [11, §6], so physical removal from a graph index may require a rebuild even though logical deletion is immediate. What consistency window is acceptable?

If per-user indexes are chosen, what is the cost at realistic user counts, and does it change the storage constraint?

References

[8] J. S. Park et al. *Generative Agents*. 2023. https://arxiv.org/abs/2304.03442

[10] J. Johnson et al. *Billion-scale Similarity Search with GPUs*. 2017. https://arxiv.org/abs/1702.08734

[11] Y. Malkov and D. Yashunin. *Efficient and Robust ANN Search using HNSW*. 2018. https://arxiv.org/abs/1603.09320